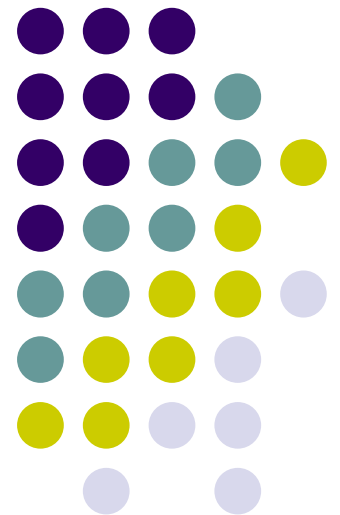


Computer Networks: Where are we?

1. Physical layer: codes
2. Datalink layer: queuing, ST
3. Network layer: fabrics, scheduling

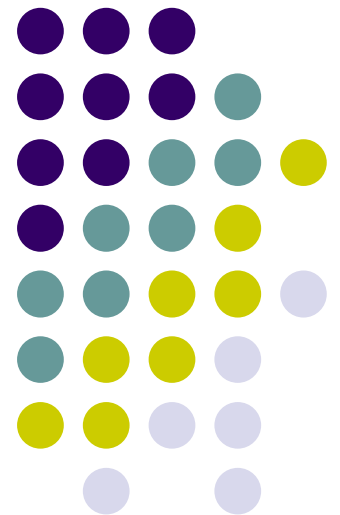


Today: Routing

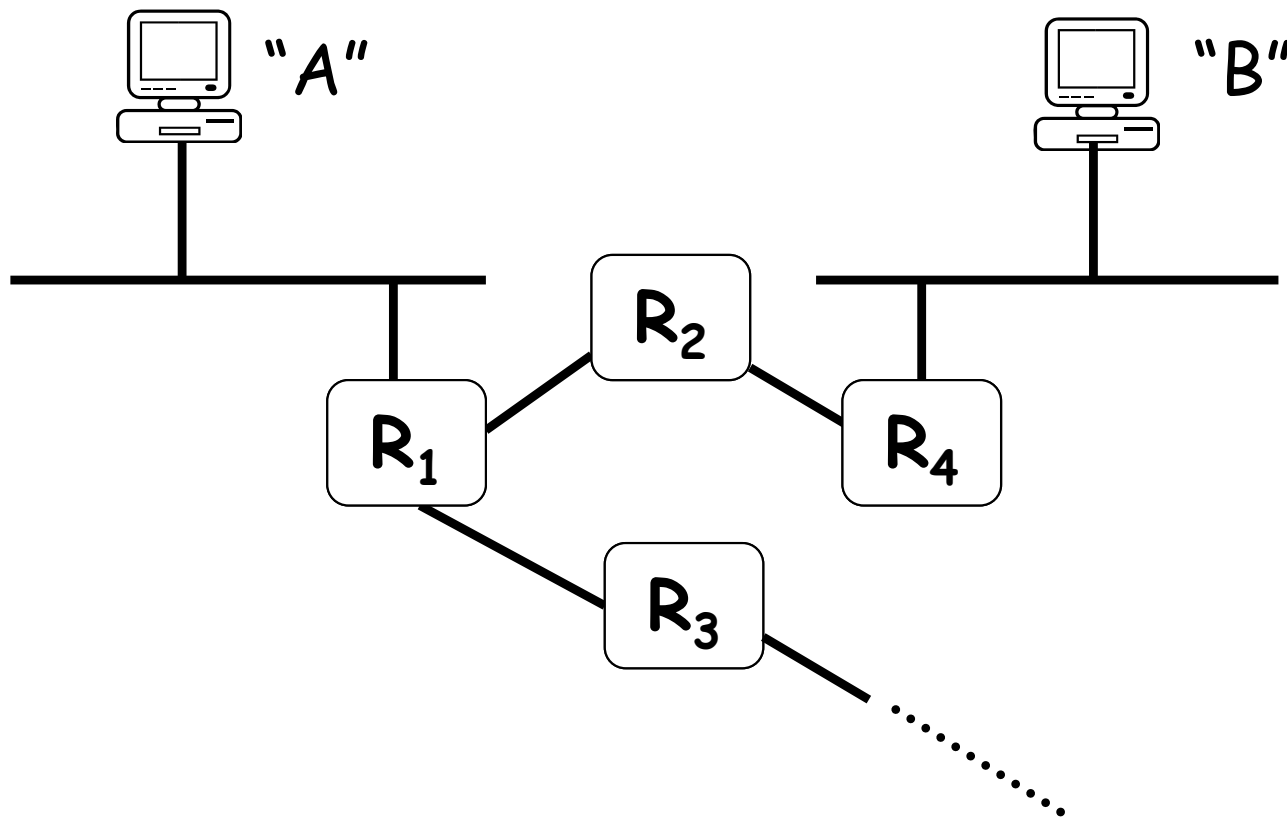
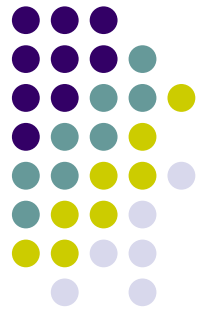
Next week: IP, forwarding

Then:

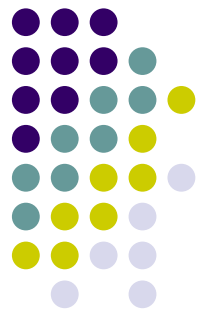
- Sliding window, TCP, security.



The Problem

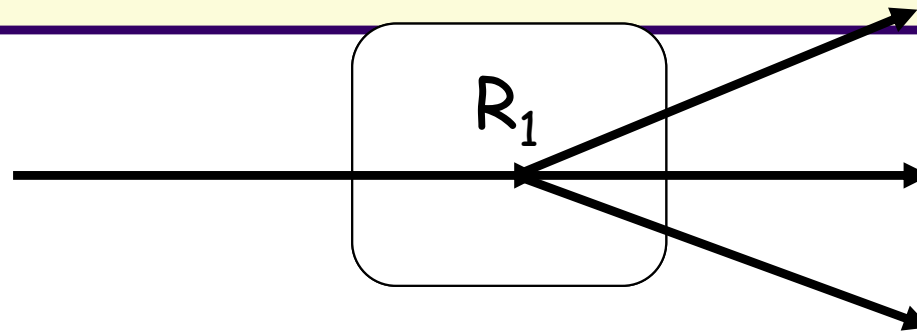


Goal: When a packet arrives with **any destination**, forward it to the right direction



Trivial solution: Flooding

Rule: Forward packets to all ports except the ingress port.



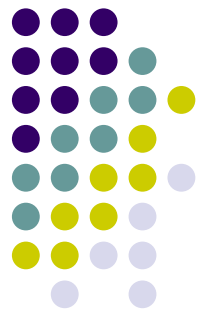
Advantages:

- ❖ Reaches destination!
- ❖ Even when network topology is completely unknown.

Disadvantages:

- ❖ Some routers receive packet multiple times.
- ❖ **Packets can go around in loops forever.**

Slightly better: *Controlled Flooding*



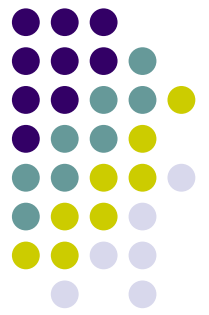
Two strategies:

1. Each message has unique ID
 - Routers remember and don't forward twice
2. Messages are time-stamped and have limited time-to-live

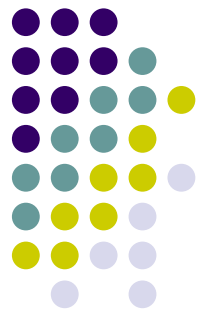
Problems:

1. Overhead: time, space
 - Each message ID must be recorded
2. Scale

Structured Approach: Forwarding/Routing decoupling



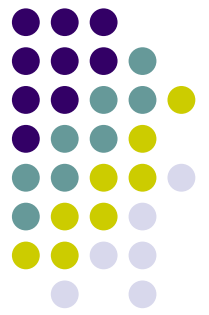
- Similar to separation of driving and navigating
- **Forwarding**: given the next hop, **move** packets from input port to output port
- **Routing**: given a packet (destination, flow...), **find** what is its next hop



Routing Tables

- Find next hop by table lookup
- Each router holds a "next hop" table: Given **any** destination, where to go next
- Challenge: How to compute these routing tables
- Routers collaboration: routing protocol





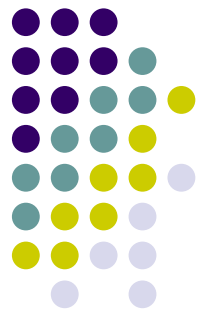
Basic Assumptions

Local knowledge:

- Router knows the status of all its incident links, and the name (address) of each neighbor

Dynamic environment:

- Dynamic network: links & routers may crash, appear
- Dynamic traffic pattern: load may change rapidly, affecting latency, bandwidth, packet drops
- Internet: alternative paths scarce (except on backbone)



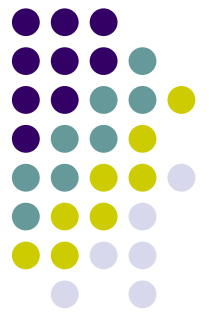
Algorithm: Bellman-Ford

Idea: build a shortest-paths tree for each destination.

Implementation:

For each destination s : each node v , periodically, tells all its neighbors what is its **distance from s**

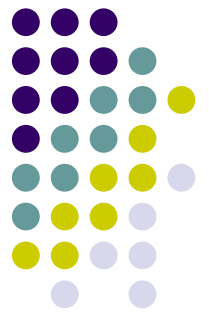
- s : easy. $d_s(s) = 0$ always!
- Another node v :
$$d_v(s) = \min\{d_u(s) + 1 \mid u \text{ is a neighbor of } v\}$$
- Mark neighbor with least distance as "next hop"
 - break ties by neighbor ID



Bellman-Ford: properties

- Works for any non-negative link weights $w(u,v)$:
$$d_v(s) = \min\{d_u(s) + w(v,u) \mid u \text{ is a neighbor of } v\}$$
- Works when the system operates asynchronously.
- Works regardless of the initial distances!

Sketch here only the synchronous, unit-value weights.

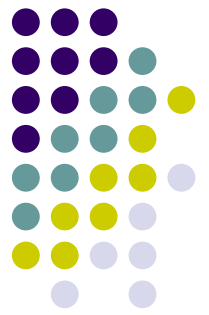


BF Algorithm: Lemma 1

$\forall v \in V, i \in \mathbb{N}$: If $\text{time} \geq i$, the distance variable in node v satisfies $d_v \geq \min(i, \text{distance}(s, v))$.

Proof: induction on time, i (all distances).

- Base: $i = 0, d_v \geq 0$ everywhere.
- Step: Suppose $d_u \geq \min(i, \text{distance}(s, u))$ for all u at time i . Consider v at time $i + 1$:
 - If v assigns more than i , then done.
 - Else v must have a neighbor u with value $d_u < i$ in step i , and by induction $\text{distance}(s, u) \leq d_u - 1$, hence $\text{distance}(s, v) \leq d_v$.

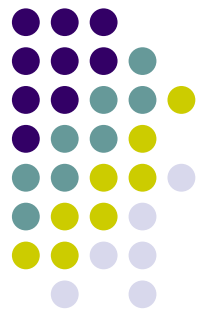


BF Algorithm: Lemma 2

$\forall v$: If $\text{time} \geq \text{distance}(s, v)$ then $d_v \leq \text{distance}(s, v)$.

Proof: By induction on distance from s (all times).

- Base: $\text{distance}(s, v) = 0$, then $s = v$, trivially $d_v \leq 0$.
- Step: Assume lemma holds for all nodes at distance d . Consider a node v at distance $d + 1$:
 - v has a neighbor u at distance d .
 - By induction, $d_u \leq d$ when $\text{time} \geq d$, hence $d_v \leq d + 1$ when $\text{time} \geq d + 1$.



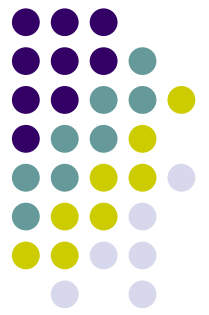
Distance Vector Algorithms

Multiple instances of (some variant of) the BF Algorithm from each node:

- each node is the source in one instance
- n instances run in parallel ($n = \text{\#nodes}$)

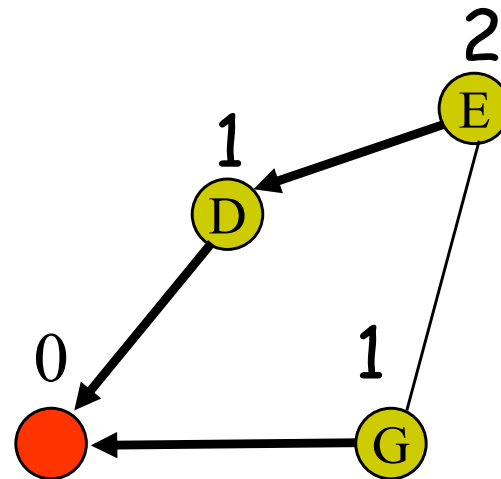
Execution: Send **vector of distances** when changes detected, or periodically (upon timeout).

- Complexity: message size $O(n)$, space $O(n)$

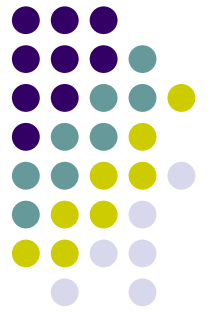


Source Crash

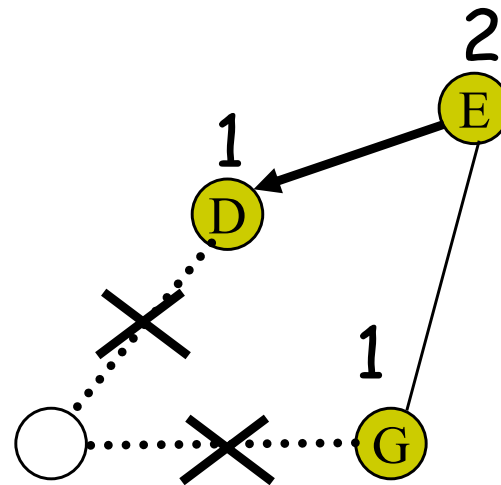
- Suppose the source becomes unconnected:

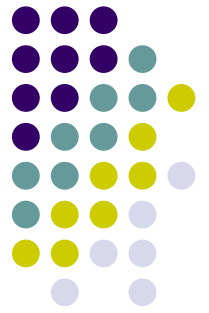


Source Crash



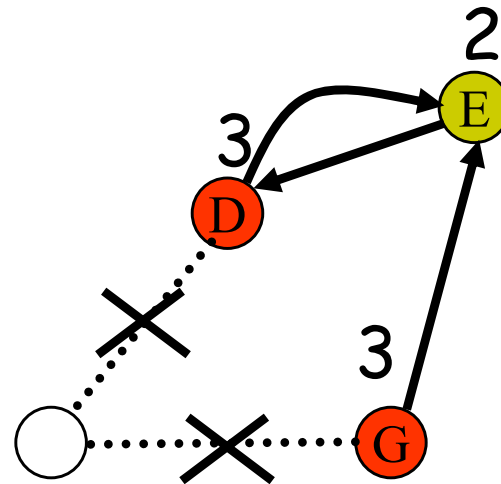
- Suppose the source becomes unconnected:

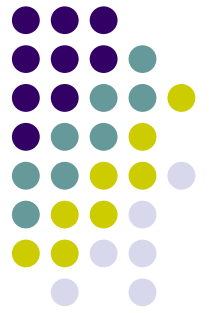




Source Crash

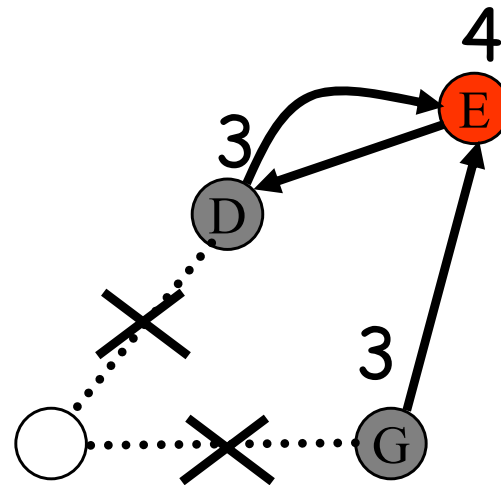
- Suppose the source becomes unconnected:

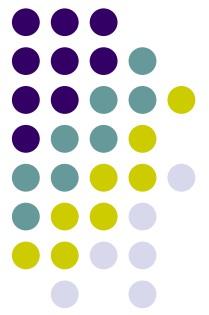




Source Crash

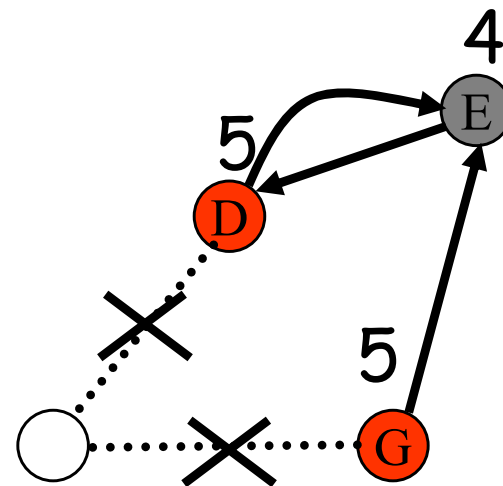
- Suppose the source becomes unconnected:





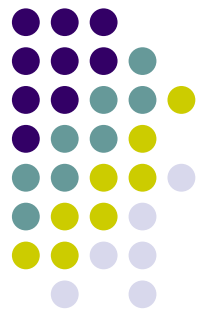
Source Crash

- Suppose the source becomes unconnected:



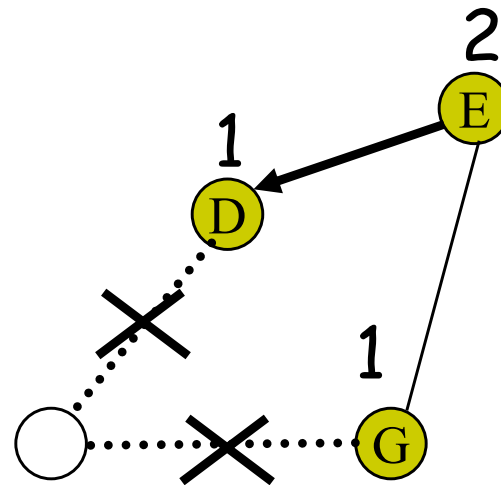
- ... and so on.

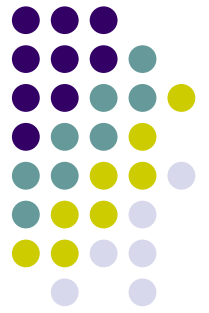
The **Counting to infinity** problem. Always happens when source crashes!



Source Crash: Split Horizon

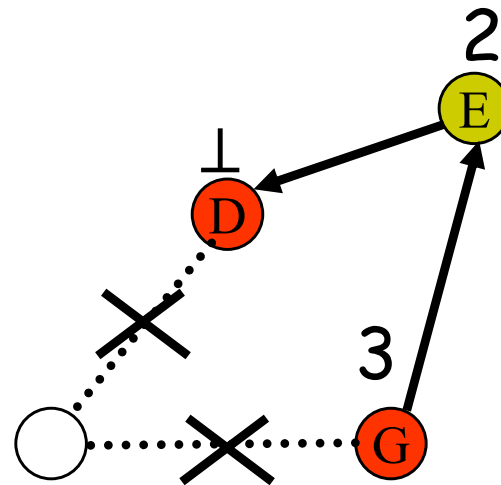
- Split horizon: do not advertise to parent
- E does not tell D

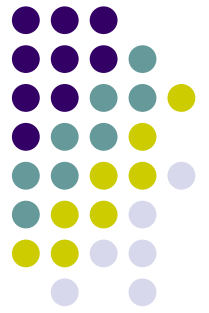




Source Crash: Split Horizon

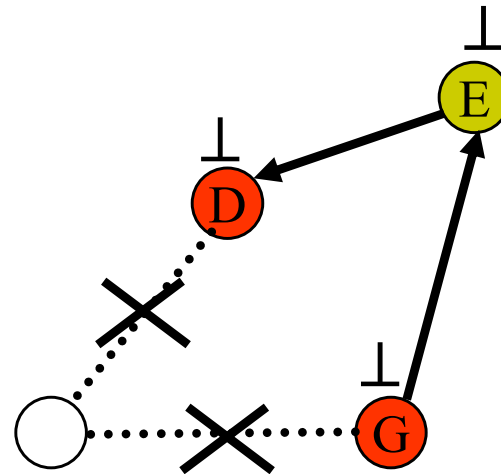
- Split horizon: do not advertise to parent
- E does not tell D

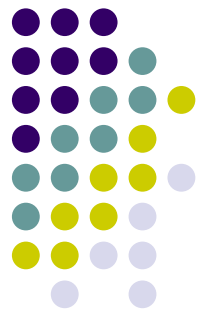




Source Crash

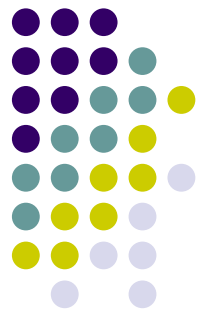
- Split horizon: do not advertise to parent
- E does not tell D
- Crashed source is erased from tables, eventually
- Not always...





Fixes

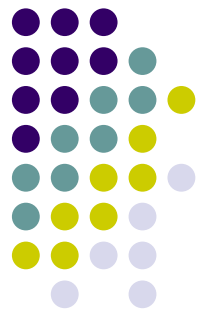
- Split Horizon: don't tell dist to parent
 - Doesn't solve the problem for larger cycles
- Add a hardwired bound on distances (TTL).
 - Bound must be small to stabilize quickly
 - Bound must be large to allow network growth.
- DUAL: when cost increases, freeze table until all affected nodes know
- **Path Vector alg.:** Advertise complete paths
 - Delete paths with cycles
 - $O(Dn)$ message size, memory space



Link State Algorithm

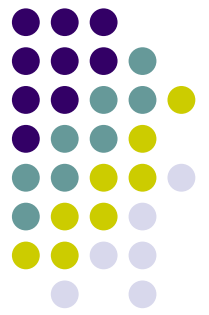
Distance vector algorithm: router knows only distances. Hides much information...

- Link state algorithm:
 - each router knows **entire network topology**.
 - Compute shortest paths **locally**
- Key elements
 1. topology dissemination: distributed protocol
 2. computing shortest paths: centralized algorithm



Link state dissemination

- A router describes its incident links using a link state packet (LSP)
- LSPs are stored in each node "database"
- Use flooding to distribute LSPs everywhere
 - LSPs have sequence numbers
 - Latest LSP stored in an LSP database
 - When a new LSP is received, it is forwarded to every interface other than incoming one
 - In a network with m links: each LSP copied at most $2m$ times (why?)

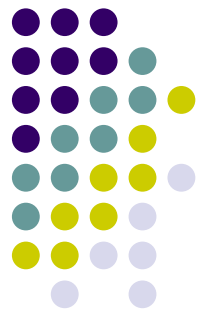


Aging

1. Creator of LSP puts timeout value in the header
2. Router removes LSP when it expires, informs others
3. On booting, router waits for its old LSPs to be purged from the network

Issue: But what age limit to choose?

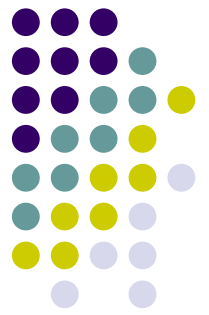
- if too small
 - purged before fully flooded
 - needs frequent updates
- if too large
 - router waits idle for a long time on rebooting



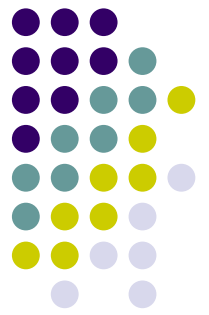
Router failure

- Easy to detect link failure
- More complicated to detect router failure
 - HELLO protocol: send HELLO every 10 seconds; if 4 messages not received, declare router dead
- HELLO packet may be corrupted
 - so age anyway
 - on a timeout, flood the information

Link State Algorithm



- ❖ Routers send out update messages whenever the state of a link changes/timeout.
- ❖ Each router calculates lowest cost path to all others, starting from itself: single source shortest paths algorithm.
- ❖ Usually: Dijkstra's shortest paths algorithm



Computing shortest paths

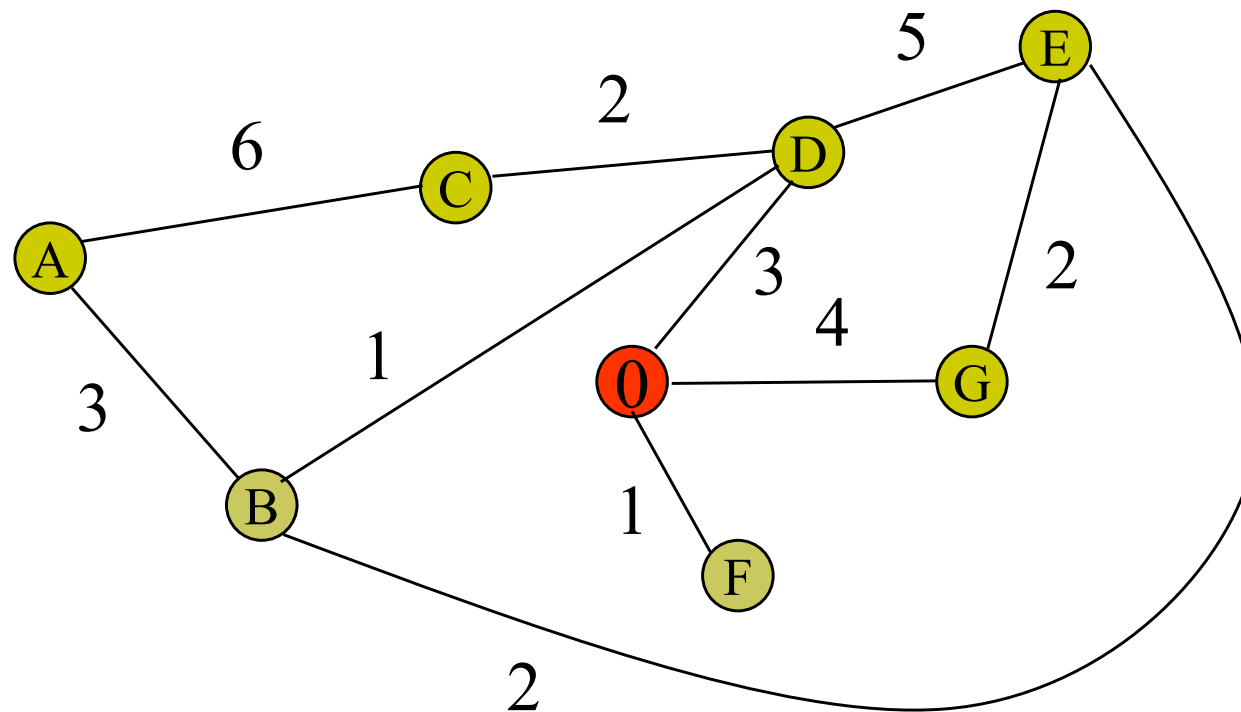
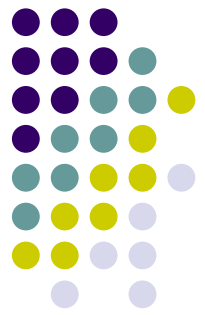
Dijkstra's Algorithm

1. Maintain a set of nodes P to whom we know shortest path, initially $P = \{\text{root}\}$
2. Find the node v closest to P : must be one of the nodes one hop away from nodes in P
3. Add v to P

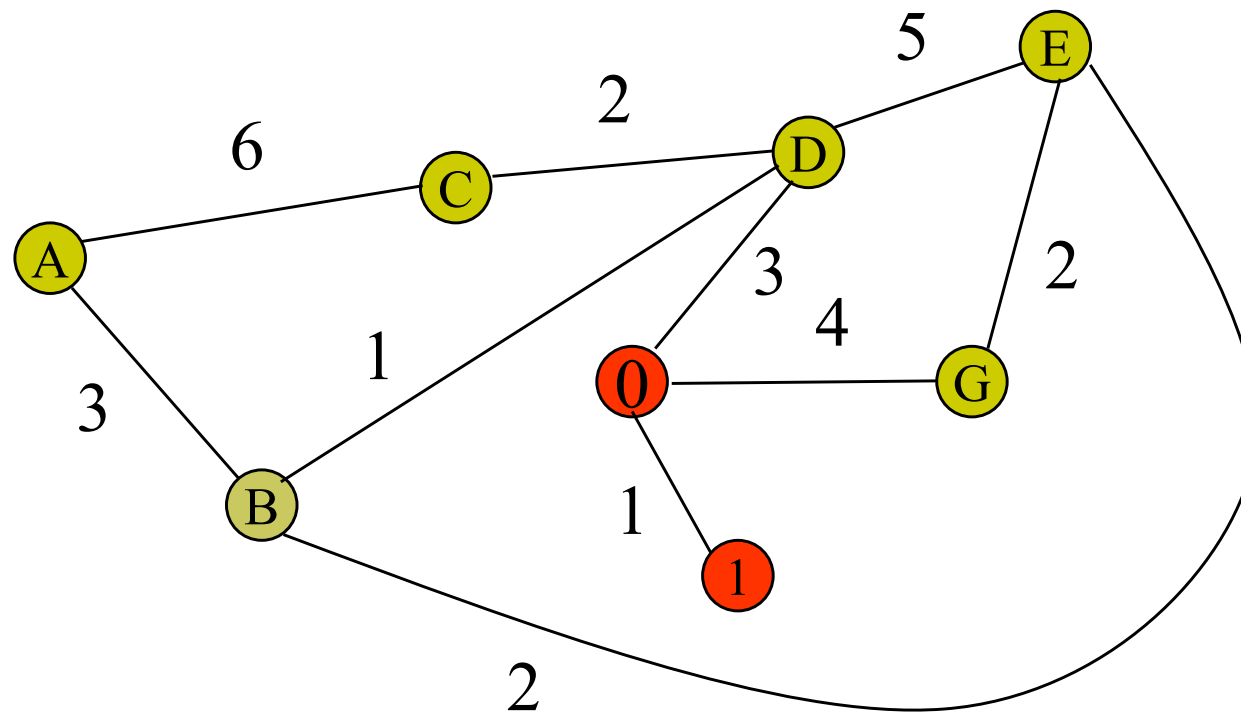
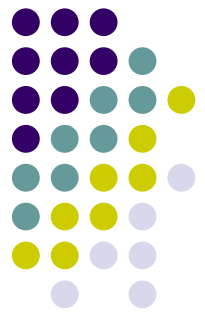
Correctness invariant: P grows in order, from the nodes closest to root to the farthest

$O(m + n \log n)$ steps for m links and n nodes

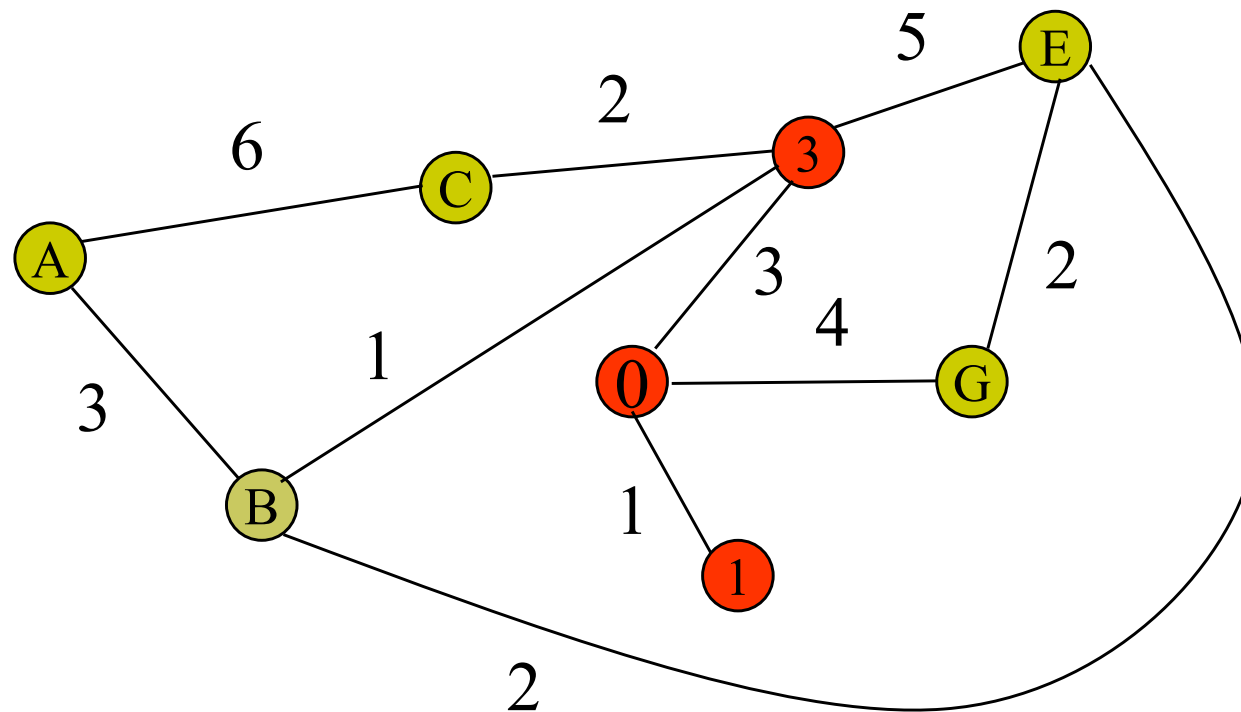
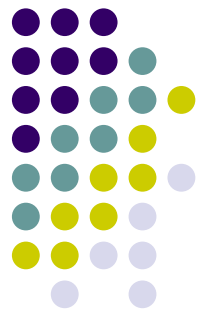
Example



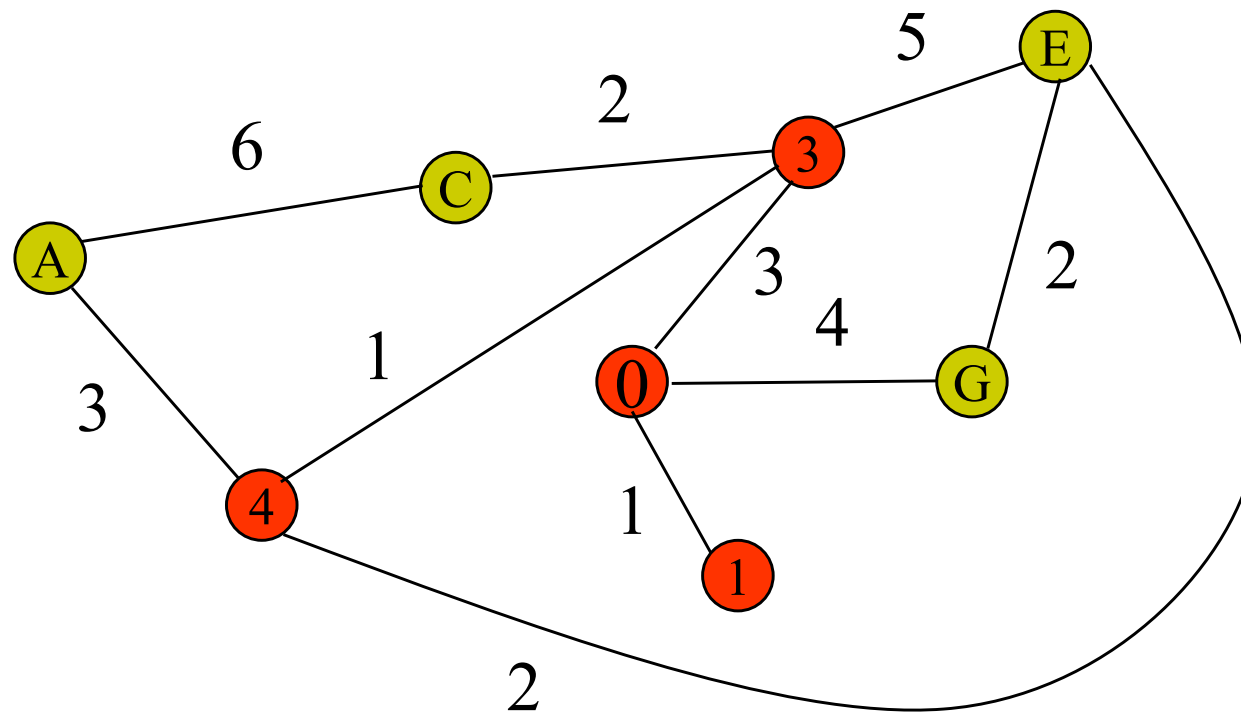
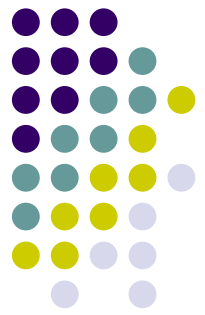
Example



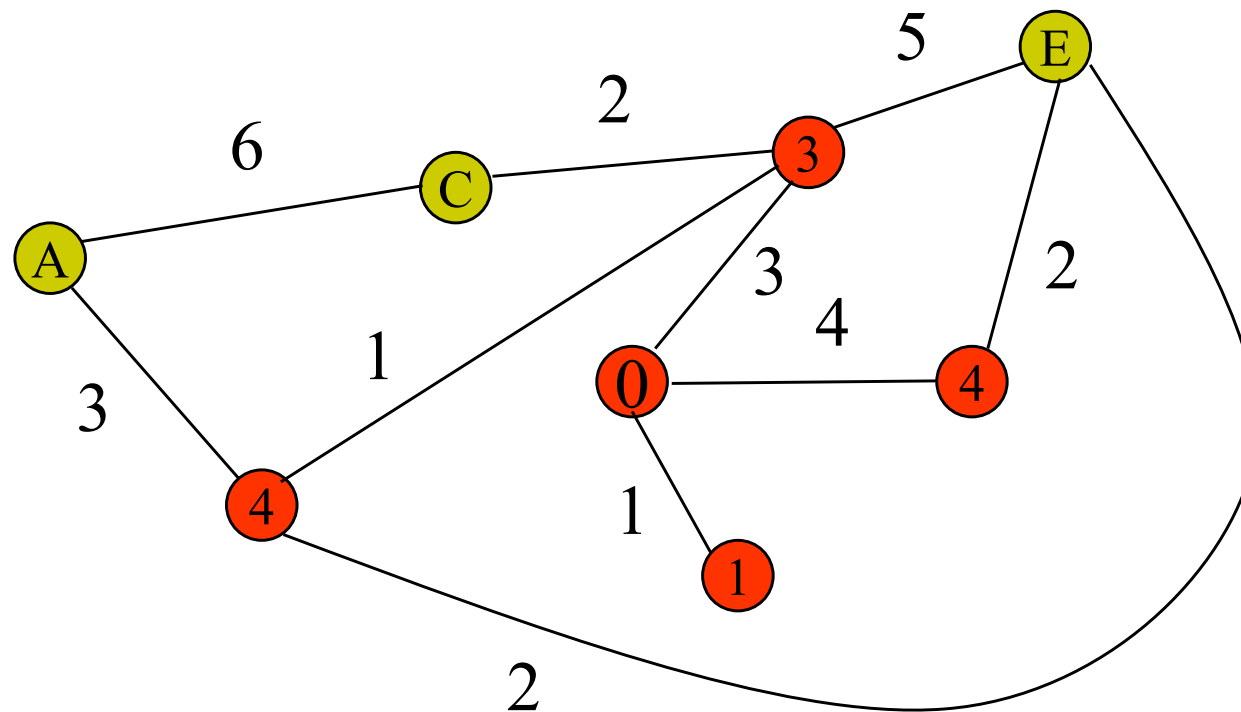
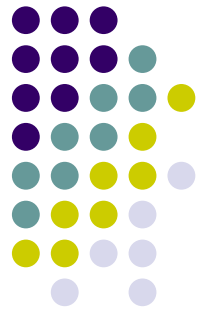
Example



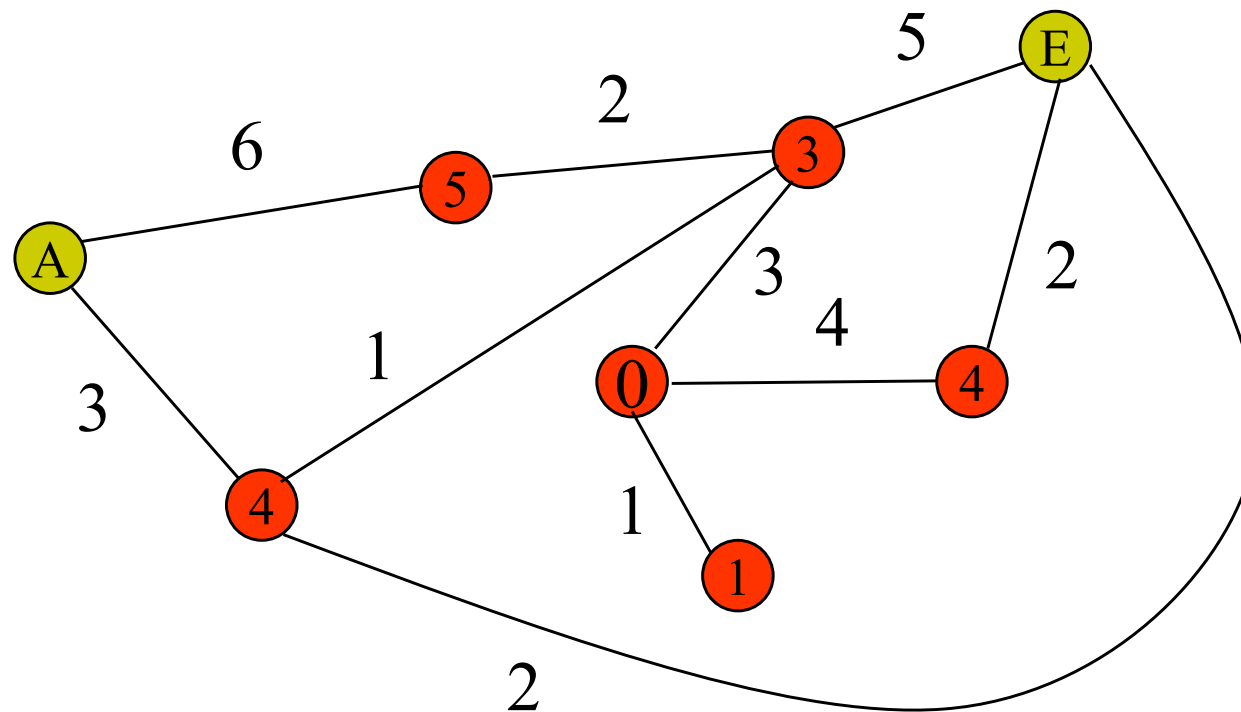
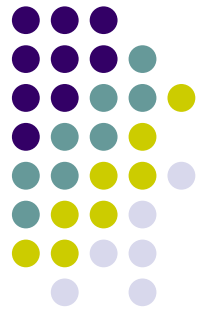
Example



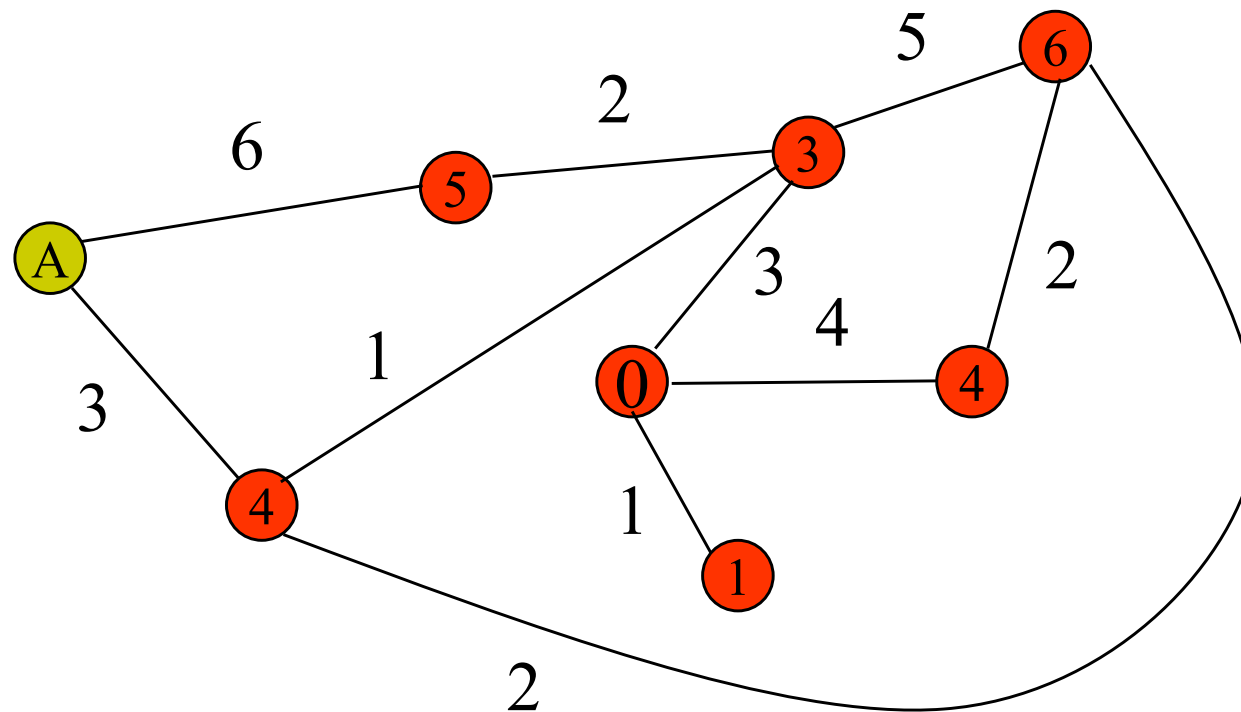
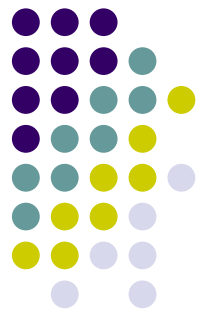
Example



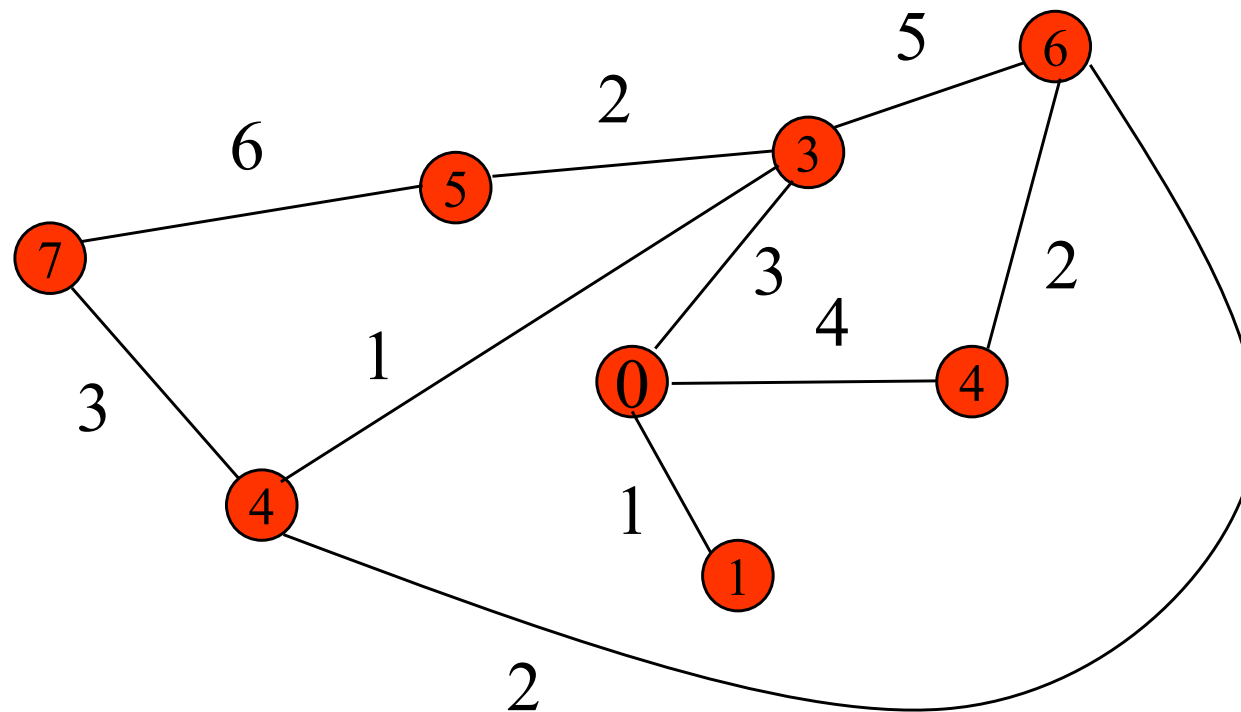
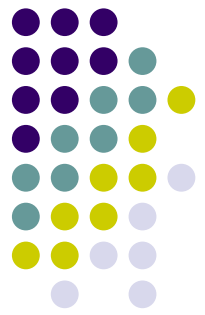
Example



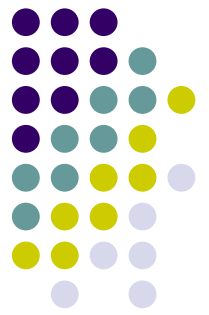
Example



Example

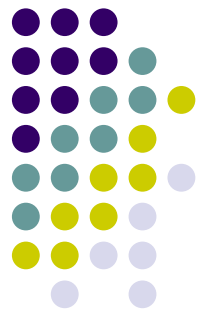


Complexity of Link state Algorithms



Let $m = \#$ links.

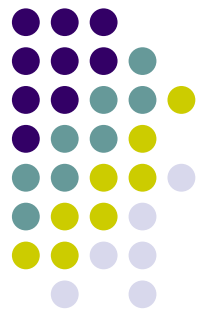
- Communication complexity: $O(m)$ messages per period
- Space complexity: $O(m)$



Distance Vector vs Link State

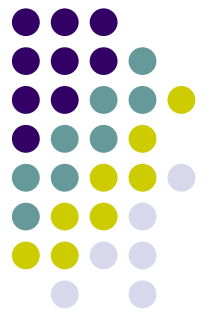
Basic ideas:

- DV: Each node advertises (sends to **neighbors**) its **distance** to each destination; choose the closest based on neighbors' announcements.
- LS: Each node advertises (floods to **all**) the state of its **incident links**; compute locally who's closest based on entire topology.



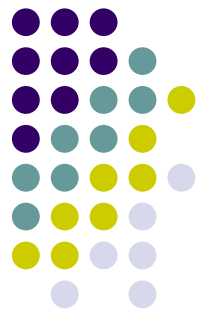
Link state vs. distance vector

- Criteria
 - Stability: LS wins?
 - Multiple routing metrics: LS wins for generality
 - Convergence time: depends on size of change
 - communication overhead: DV wins
 - memory overhead: DV wins
- Both are evenly matched
- Both widely used



Choosing link costs

- Shortest path uses link costs
- Possible costs criteria:
 - Time
 - Bandwidth
 - Reliability
- Usually: cost determines amount of traffic on the link
 - lower the cost, more the expected traffic
 - if dynamic cost depends on load, can have oscillations (why?)

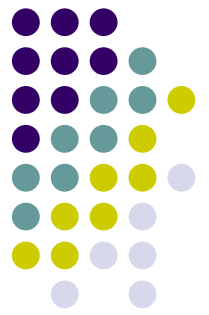


Static metrics

- Simplest: set all link costs to 1. Min hop routing
 - Propagation delay is roughly the same on all media
 - but 1.5Mbps ADSL link $\neq$ Gigabit ethernet!
- Solution: assign cost inversely proportional to bandwidth

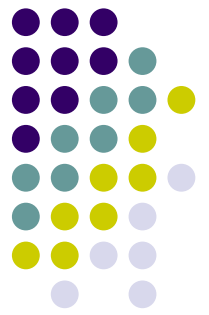
$$\text{cost} = \left\lceil \frac{\text{reference bw}}{\text{link bw}} \right\rceil$$

- Actually, most time is spent waiting in queues...



Dynamic metrics: time varying

- ARPAnet original: Cost proportional to queue length, independent of link capacity
- Many problems when network is heavily loaded...
 - transient spikes cause major rerouting
 - network ignores high-cost paths
 - Oscillations between paths of similar costs
 - all tables computed simultaneously => low cost links overwhelmed



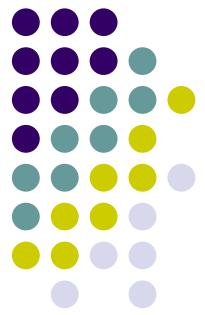
Modified metrics

Old ARPANET

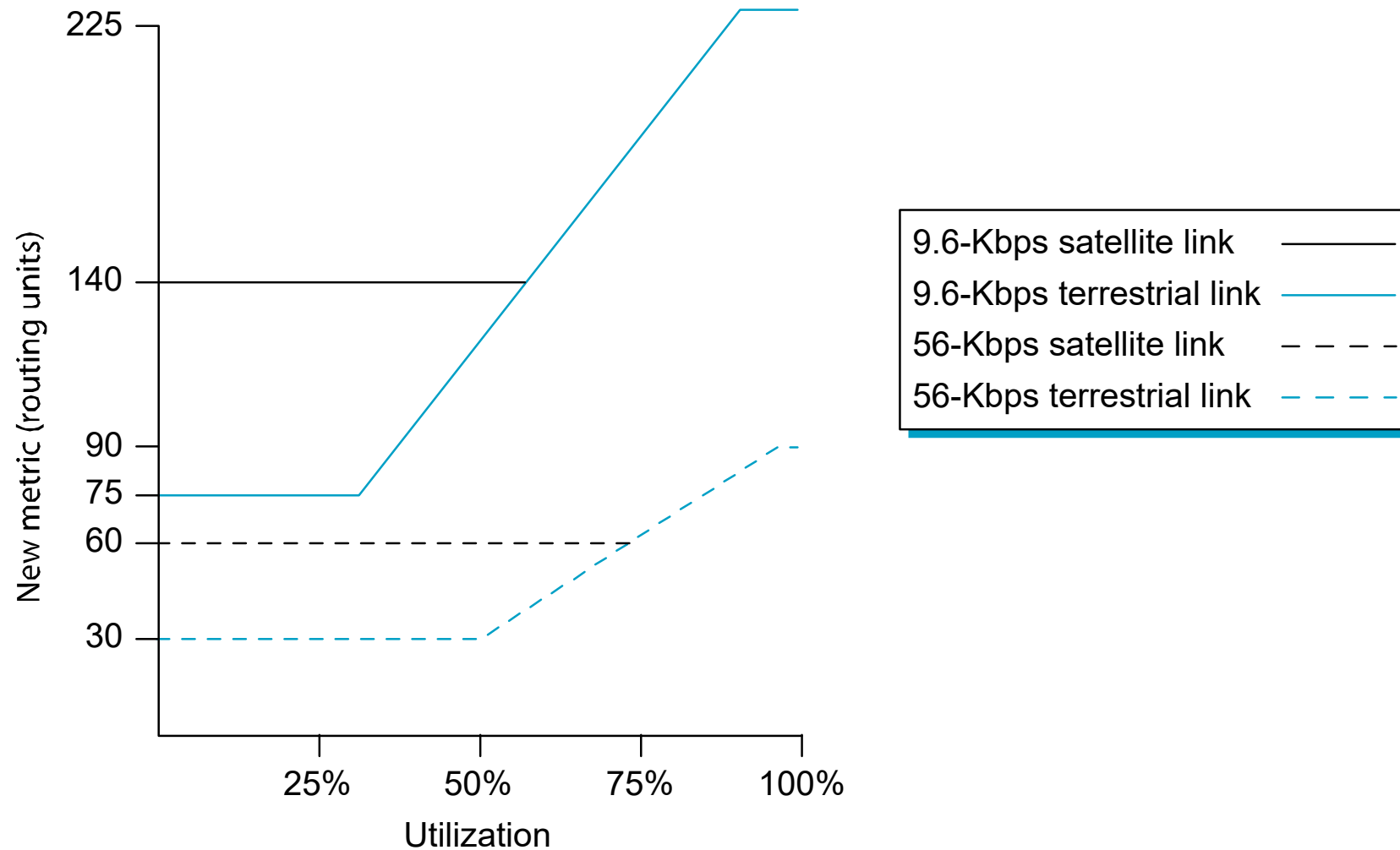
- queue length averaged over a small time
- wide dynamic range
- queue length assumed to predict future loads
- no restriction on successively reported costs
- all tables computed simultaneously

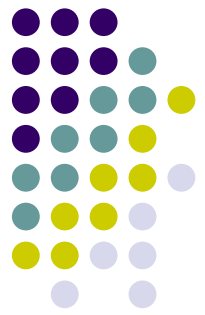
New metrics

- queue length averaged over a longer time
- dynamic range restricted
- cost also depends on intrinsic link capacity
- restriction on successively reported costs
- attempt to stagger table computation



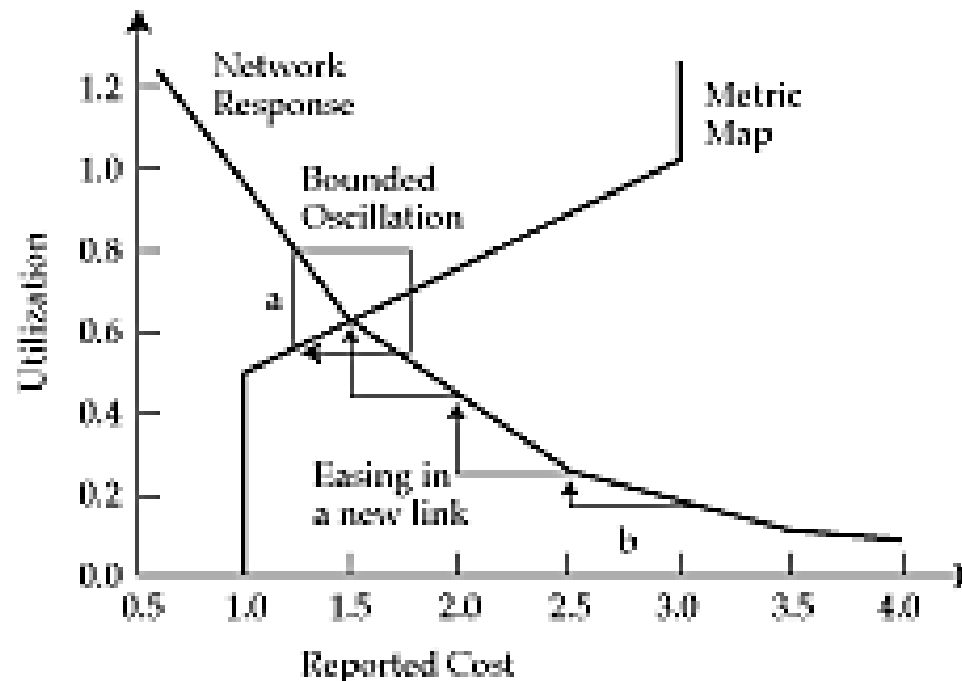
Example costs

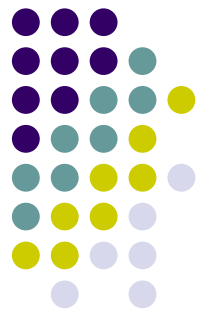




Routing dynamics

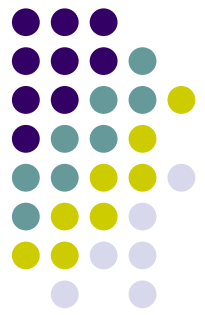
- If a link is lightly loaded, its cost is small
 - it may become part of a shortest path
 - it get much traffic and its cost increases





Hierarchical routing

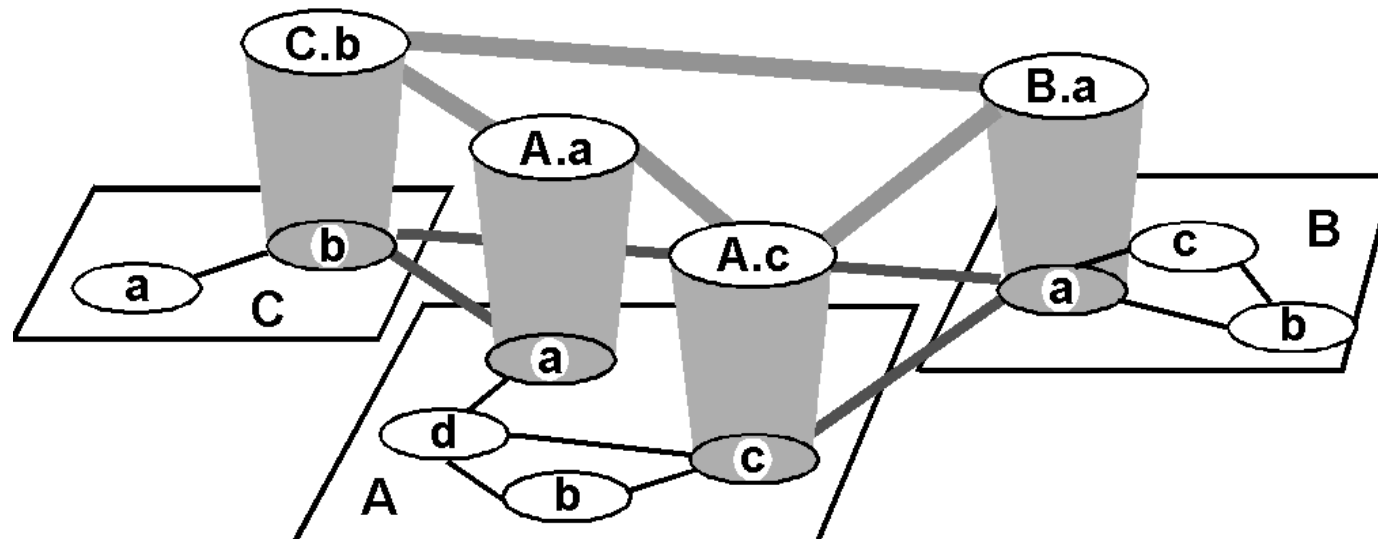
- Large networks need large routing tables
 - more computation to find shortest paths
 - more bandwidth wasted on exchanging DVs and LSPs
- Solution:
 - hierarchical routing
- Key idea
 - divide network into a set of **domains**
 - **gateways** connect domains
 - computers within domain unaware of outside computers
 - gateways know only about other gateways

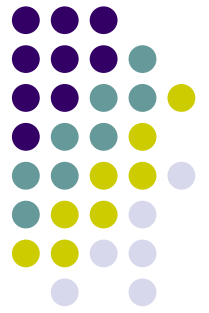


Example

Features

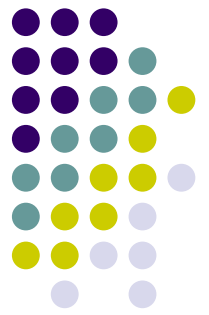
- only a few routers in each level
- not a strict hierarchy
- gateways participate in multiple routing protocols
- non-aggregable routers increase core table space





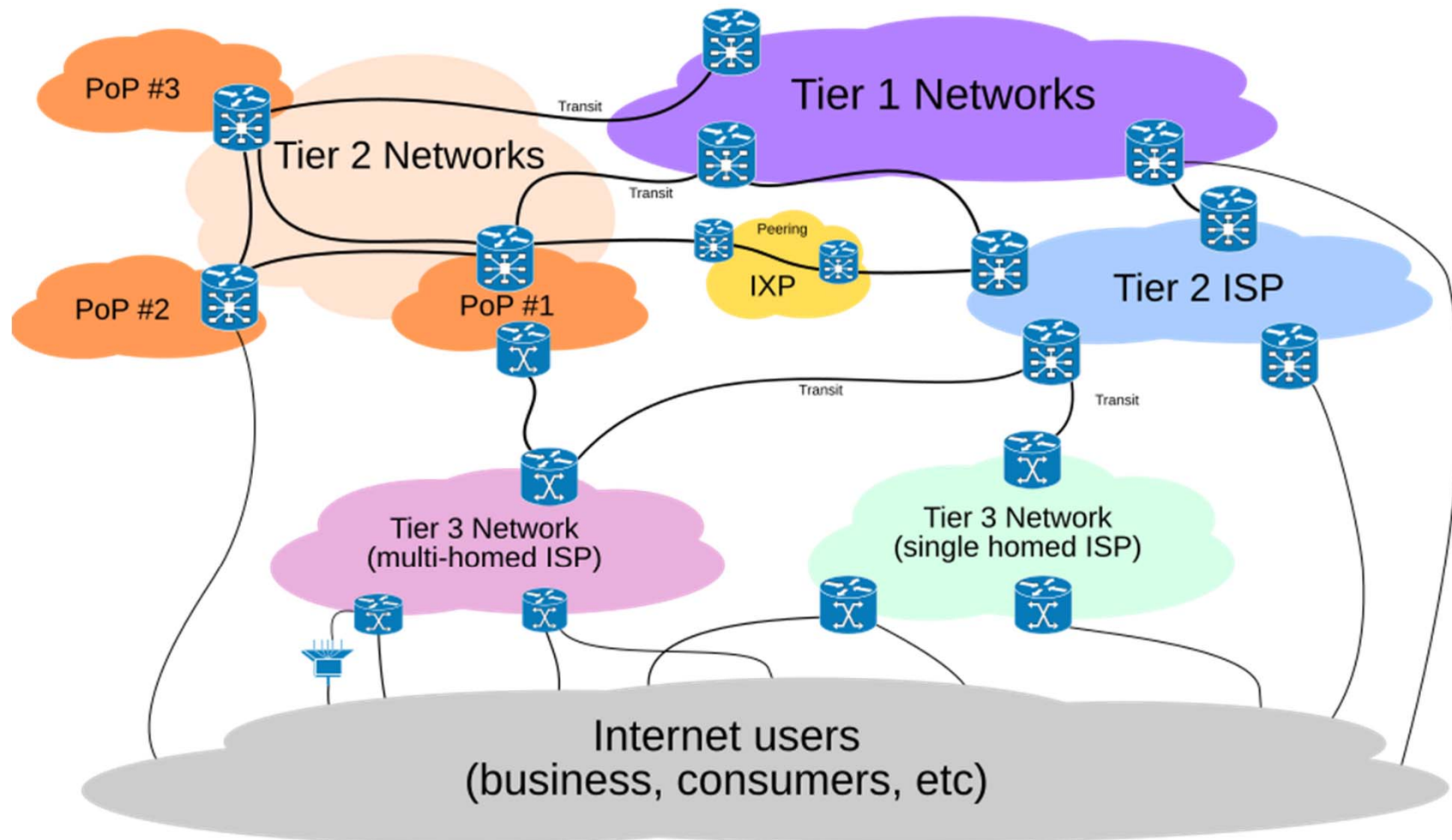
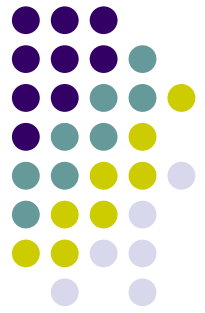
Routing in the Internet

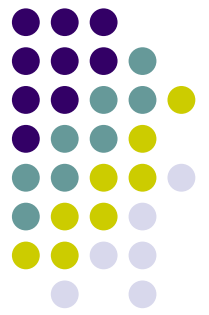
Hierarchical routing in the Internet



- Two levels of routing:
 - Organization are called autonomous systems (**AS**). Each AS implements its own routing
 - *Backbone level, connecting AS's*
- Protocol between AS gateways: exterior gateway protocol
- Protocol within AS: interior gateway protocol

Internet AS's

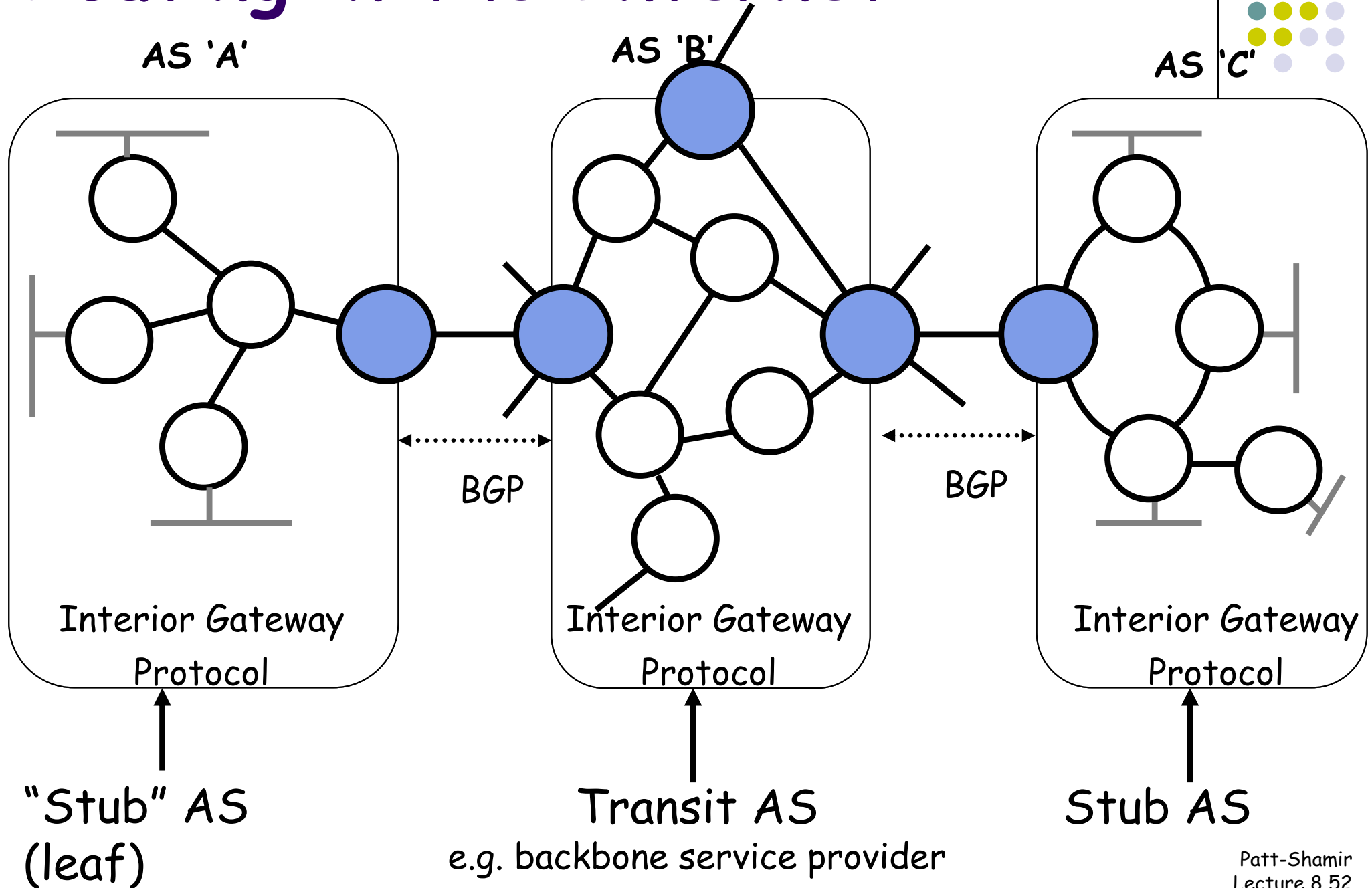
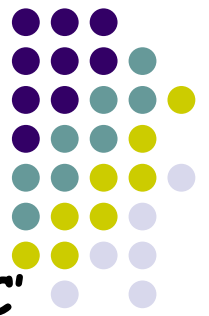


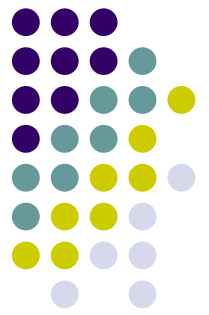


Internet routing protocols

- Interior
 - RIP: distance vector
 - OSPF: link-state
 - IGRP (EIGRP): distance vector
 - ... Others also possible
- Exterior: must have exactly one!
 - BGP: path vector

Routing in the Internet

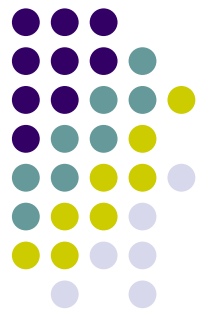




Routing within a Stub AS

- ❖ There is only one exit point (usually*), so routers within the AS can use *default routing*.
- ❖ Each router knows all network IDs within AS.
- ❖ Packets destined to another AS are sent to the default router.
- ❖ Default router is the **gateway** to the next AS.
- Routing tables in Stub AS's tend to be small.

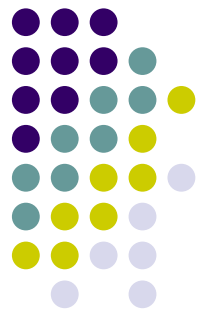
* Large AS's may have more than one ISP: multihomed



RIP

Routing Information Protocol (**routed** in BSD4.2, 1982)

- Distance vector
 - unweighted B-F (cost=hop count)
 - max #hops (TTL) = 16
 - “Split horizon with poisonous reverse”: tell your next hop neighbor to s that your distance to s is ∞
 - Exchange distance vectors every 30 s. Silence for 180 seconds interpreted as crash.
- Useful and popular for small networks
 - easy to install

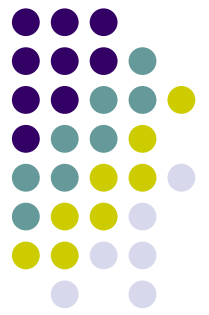


OSPF

Open Shortest Paths First (1989)

Link-state algorithm with advanced features:

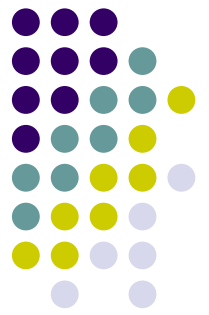
- May use multiple paths to balance load
- Possibly various metrics
- Hierarchical:
 - Defines **areas** with “autonomous” routing
 - Uses **area border routers** for next level in hierarchy
 - Makes each routing table smaller
- Secure: LSPs are authenticated, databases are protected
- Complex



IGRP => EIGRP

(Extended) Interior Gateway Routing Protocol

- CISCO proprietary (1997); successor of RIP
- Distance Vector, like RIP
- Several cost metrics (delay, bandwidth, reliability, load etc)
- Routing tables exchanged only when costs change
- Loop-free routing achieved by DUAL

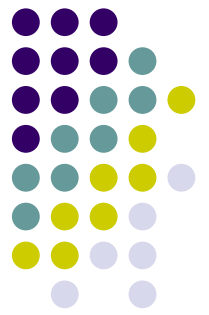


Exterior Routing Protocols

What we have to work with:

- ❖ Incompatibility of AS's link costs
- ❖ Different AS's have different objectives
- ❖ Some AS's don't trust others

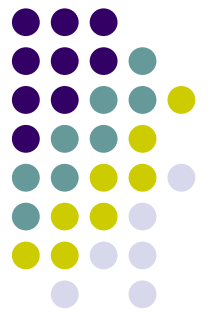
Internet inter-AS routing: BGP



Path Vector protocol:

- Similar to Distance Vector protocol, but each node broadcasts to neighbors (peers) *entire path* (i.e, sequence of ASs) to destination
- E.g., node X may send its path to dest. Z :

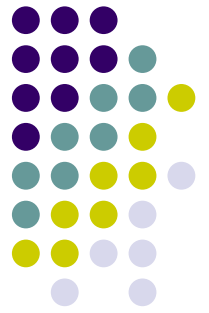
$$\text{Path}(X, Z) = X, Y_1, Y_2, Y_3, \dots, Z$$



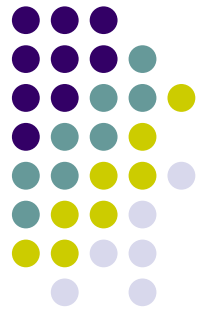
BGP Rationale

- Nodes may select paths based on their own discretion, e.g.
 - cost, payment, policy (don't route via competitors AS), loop prevention reasons.
- A Node can control incoming traffic by controlling its route advertisements to peers

BGP

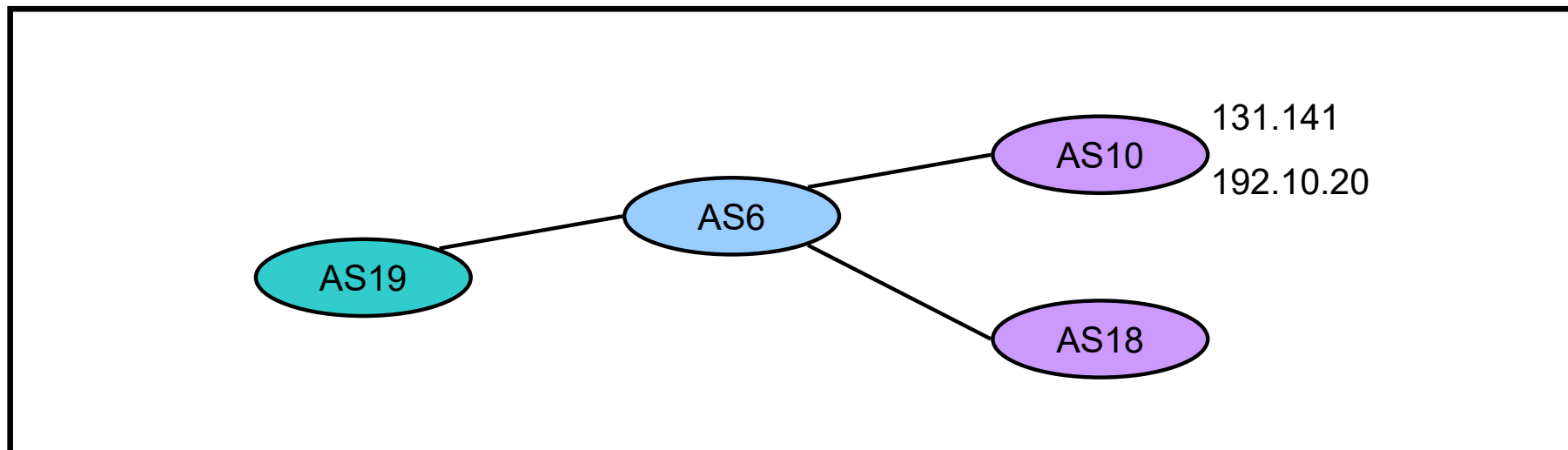


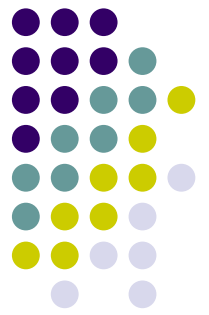
- Each AS has
 - At least one border gateway to handle inter-AS traffic
 - At least one router that speaks BGP or a node that participates in the routing protocol
- BGP speakers advertise
 - Names of networks within the AS
 - Names of networks reachable through the AS
 - Full path information
 - Intradomain protocols use heterogeneous metrics
 - Withdrawn routes/negative advertisements



BGP-4

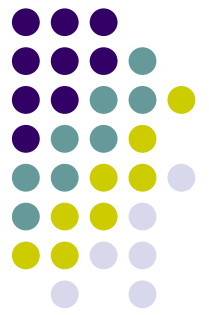
- Path-vector routing: complete paths
 - Guarantees loop-free
- Example
 - AS10 advertises 131.141 and 192.10.20 as local networks
 - AS6 advertises same networks with path through (AS10)
 - AS19 advertises same networks with path through (AS6,AS10)





BGP implementation

- Extremely complex!
- BGP messages exchanged using TCP.
- BGP messages:
 - **OPEN**: opens TCP connection to peer and authenticates sender
 - **UPDATE**: advertises new path (or withdraws old)
 - **KEEPALIVE** keeps connection alive in absence of UPDATES; also ACKs OPEN request
 - **NOTIFICATION**: reports errors in previous msg; also used to close connection



New Concepts

- Flooding, controlled
- Distance vector alg.
- Counting to infinity
 - Split horizon
 - TTL
 - Path vector alg.
- Link state alg.
- Link state packet
- Hierarchical routing
- Autonomous system
- Internal/exterior routing protocol
- RIP, OSPF, EIGRP, BGP